

■ 강의명: CSCI103

■ 주차: Lecture 13

■ 교수명: UIUC Student

■ 목적: 본 문서는 CSCI103 Lecture 13 강의 내용을 바탕으로, 데이터 이니셔티브 투자에서 최대 가치를 얻고 지속적인 개선을 달성하기 위한 핵심 전략을 다룹니다. 최종 과제 안내를 시작으로, 소프트웨어 개발 수명 주기(SDLC)의 각 단계와 견고하고 확장 가능한 데이터 제품 및 서비스 구축 방법을 설명합니다. 특히, 서비스 수준 계약(SLA)의 중요성과 비용 절감 전략을 상세히 다루며, 자동화된 인프라 관리를 위한 지속적인 통합 및 배포(CI/CD)와 Infrastructure as Code(IaC) 개념을 심층적으로 탐구합니다. Databricks 플랫폼의 워크스페이스 조직, 데이터 메시, 그리고 관측 가능성(Observability) 및 모니터링(시스템 테이블, 이상 감지, 레이크하우스 모니터링, DQX) 기능이 소개됩니다. 마지막으로, Databricks Asset Bundles(DABs)를 활용한 프로젝트 아티팩트 관리 및 CI/CD 통합 방안을 실습 예시와 함께 제시하여, 데이터 플랫폼 운영의 효율성과 안정성을 극대화하는 방법을 종합적으로 이해할 수 있도록 돕습니다.

Contents

1 용어 정리	2
2 최종 과제 안내	3
2.1 발표 형식 및 내용	3
2.2 발표 지침	3
3 데이터 파이프라인 운영 및 최적화	4
3.1 견고하고 확장 가능한 데이터 제품 및 서비스 구축	4
3.2 소프트웨어 개발 수명 주기 (SDLC) 단계	5
3.3 서비스 수준 계약 (SLA)	5
3.4 비용 절감 전략	6
4 지속적인 통합 및 배포 (CI/CD)	8
4.1 CI/CD의 기본 개념	8
4.2 데이터 플랫폼에서의 CI/CD	8
5 인프라 자동화: Infrastructure as Code (IaC)	10
5.1 IaC의 핵심 원칙	10
5.2 자동화 도구 비교: 구성 관리 vs. 프로비저닝	10
5.3 Terraform 데모 예시	11
6 워크스페이스 조직 및 데이터 메시	12
6.1 워크스페이스 조직 (Workspace Organization)	12
6.2 데이터 메시 (Data Mesh)	12
7 Databricks에서의 CI/CD 및 MLOps	14
7.1 CI/CD 파이프라인 (Azure DevOps 예시)	14
7.2 MLOps (Machine Learning Operations)	14

8 관측 가능성 (Observability) 및 모니터링	16
8.1 시스템 테이블 (System Tables)	16
8.2 대시보드 (Dashboards)	16

1 용어 정리

Table 1: 주요 용어 및 설명

용어	쉬운 설명	원어	비고
SDLC	소프트웨어 개발의 전 과정	Software Development Life Cycle	개발, 스테이징, 프로덕션 단계 포함
SLA	서비스 제공자와 사용자 간의 성능 약속	Service Level Agreement	가용성, 성능, 응답 시간 등
CI/CD	코드 변경 사항을 자동으로 통합하고 배포하는 과정	Continuous Integration/Continuous Delivery	개발 효율성 및 배포 속도 향상
IaC	인프라를 코드로 정의하고 관리하는 방법	Infrastructure as Code	반복 가능하고 일관된 인프라 구축
RTO	재해 발생 후 시스템 복구 목표 시간	Recovery Time Objective	비즈니스 연속성 계획의 핵심 지표
RPO	재해 발생 시 데이터 손실 허용 최대 시간	Recovery Point Objective	백업 및 복구 전략 수립에 중요
Data Mesh	데이터를 제품처럼 취급하는 분산 아키텍처	Data Mesh	데이터 도메인 중심의 데이터 관리
MLOps	머신러닝 모델의 개발부터 배포, 운영까지의 과정	Machine Learning Operations	ML 모델의 CI/CD 및 모니터링
Observability	시스템 내부 상태를 외부에서 추론할 수 있는 능력	Observability	모니터링보다 포괄적인 개념
Databricks Asset Bundles (DABs)	Databricks 프로젝트 자산을 패키징하고 배포하는 도구	Databricks Asset Bundles	CI/CD를 위한 프로젝트 단위 배포
Unity Catalog	Databricks의 통합 데이터 거버넌스 솔루션	Unity Catalog	데이터, ML 모델, 노트북 등에 대한 접근 제어
TPC-DS	분석 및 의사결정 지원 시스템 성능 벤치마크	Transaction Processing Performance Council - Decision Support	데이터 웨어하우스 쿼리 속도 측정
TPC-DI	데이터 통합 시스템 성능 벤치마크	Transaction Processing Performance Council - Data Integration	데이터 웨어하우스 로딩 속도 측정
Spot Instances	클라우드 제공자의 유휴 컴퓨팅 자원	Spot Instances	저렴하지만 취소될 수 있는 인스턴스
Serverless	서버 관리 없이 코드 실행에만 집중하는 컴퓨팅 모델	Serverless Computing	사용량 기반 과금, 빠른 시작
Wheel Files	Python 패키지 배포 형식	Wheel Files	공유 라이브러리 빌드 및 재사용
Anomaly Detection	데이터에서 비정상적인 패턴을 자동으로 감지하는 기능	Anomaly Detection	AI 기반 데이터 품질 모니터링
Lakehouse Monitoring	Databricks에서 테이블 중심으로 데이터 품질을 모니터링하는 기능	Lakehouse Monitoring	프로파일링, 드리프트 감지 포함
DQX	Databricks Labs에서 개발한 코드 기반 데이터 품질 프레임워크	Databricks Quality eXpectations	PySpark 파이프라인 내 품질 검사

2 최종 과제 안내

개념

최종 과제 개요 최종 과제는 팀 프로젝트로 진행되며, 데이터 이니셔티브의 가치를 극대화하고 지속적인 개선을 목표로 합니다. 팀원들과 협력하여 데이터 제품 및 서비스를 개발하고, 그 과정과 결과를 발표해야 합니다.

최종 과제는 지난 밤에 Canvas에 공개되었으며, Canvas의 'People' 페이지에서 새로운 팀을 확인할 수 있습니다. 팀 프로젝트이므로, 팀원들에게 가능한 한 빨리 연락하여 소개하고 원활한 의사소통을 시작하는 것이 중요합니다. 연휴 기간이지만 상당한 작업량이 예상되므로, 충분한 시간을 확보하여 과제를 수행해야 합니다.

2.1 발표 형식 및 내용

각 팀은 최종 강의일(15일)에 발표를 진행합니다. 발표에는 다음 내용이 포함되어야 합니다.

1. **문제 정의 (Problem Statement):** 해결하고자 하는 비즈니스 문제 또는 데이터 관련 과제를 명확히 제시합니다.
2. **팀 소개 (Team Introduction):** 각 팀원은 자신의 역할을 맡아 소개합니다. 팀 규모를 고려할 때, 각 역할은 두 명의 팀원으로 구성될 수 있습니다.
3. **설계 및 아키텍처 검토 (Design and Architecture Review):** 데이터 아키텍트가 담당하며, 확장성 (Scalability), 품질 (Quality), 성능 (Performance), 신뢰성 (Reliability), 거버넌스 (Governance) 측면을 고려하여 설계 및 아키텍처를 설명합니다.
4. **데이터 파이프라인 검토 (Data Pipeline Review):** 데이터 엔지니어가 담당하며, 데이터 수집, 변환, 로딩 과정을 설명합니다.
5. **ML 모델 검토 (ML Model Review):** ML 실무자가 담당하며, 모델 개발 과정과 결과를 설명합니다.
6. **BI 대시보드 검토 (BI Dashboard Review):** 데이터 분석가가 담당하며, 비즈니스 인텔리전스 대시보드를 시연하고 주요 인사이트를 요약합니다.
7. **인사이트 요약 및 다음 단계 (Summarize Insights and Next Steps):** 프로젝트를 통해 얻은 주요 인사이트를 요약하고, 향후 개선 또는 확장 계획을 제시합니다.

2.2 발표 지침

- **슬라이드 수:** 20장 이내로 제한합니다.
- **발표 시간:** 15분 발표, 5분 Q&A로 총 20분입니다.
- **전원 발표:** 모든 팀원이 발표에 참여해야 합니다. 한 명이 슬라이드 쇼를 진행할 수 있지만, 모든 팀원이 발표의 일부를 맡아야 합니다.
- **외부 게스트:** 외부 게스트가 참석하여 발표를 검토하고 피드백을 제공할 예정입니다.
- **팀 이름:** 팀 이름을 정해야 합니다.
- **발표 순서:** 기본적으로 그룹 번호 순서로 진행되지만, 팀원 중 다른 시간대에 거주하여 늦은 저녁 시간이 되는 경우 우선적으로 발표할 수 있도록 예외를 적용합니다. 해당되는 경우 미리 알려주세요.

3 데이터 파이프라인 운영 및 최적화

개념

데이터 파이프라인 운영 및 최적화 목표 데이터 파이프라인은 비즈니스에 가치를 제공하는 핵심 요소입니다. 이를 효과적으로 운영하고 최적화하기 위해서는 초기 설계 단계부터 확장성, 효율성, 보안을 고려해야 하며, 지속적인 모니터링과 개선을 통해 안정적인 서비스를 제공해야 합니다.

3.1 견고하고 확장 가능한 데이터 제품 및 서비스 구축

데이터 파이프라인을 구축할 때 고려해야 할 중요한 사항들은 다음과 같습니다.

1. **사용 사례 이해:** 비즈니스 과제와 도메인별 특성을 깊이 이해하는 것이 중요합니다. 파이프라인이 해결하고자 하는 실제 문제를 명확히 파악해야 합니다.
2. **요구사항 정의:** 기능적 요구사항(Functional Requirements)과 비기능적 요구사항(Non-functional Requirements)을 모두 명확히 정의해야 합니다. 특히 성능(Performance) 및 확장성(Scale)과 관련된 서비스 수준 계약(SLA)을 고려해야 합니다.
3. **확장성을 고려한 구축 (Build for Scale):** 초기에 소규모로 작동하는 시스템을 구축한 후, 실제 부하가 가해졌을 때 시스템이 무너지는 경우가 많습니다. 이는 전체 프로젝트를 위험에 빠뜨릴 수 있으므로, 모델을 소비할 사용자 또는 소비자의 수를 미리 고려하여 적절한 확장을 지원할 수 있도록 설계해야 합니다.
4. **반복 가능한 패턴 및 프레임워크 사용:** 구성(Configuration) 기반의 반복 가능한 패턴과 프레임워크를 사용하여 일관성과 효율성을 높입니다.
5. **모범 사례 문서화:** 팀 내부는 물론 조직 전체의 다른 팀을 위해서도 모범 사례를 문서화하여 지식 공유와 일관된 개발을 촉진합니다.
6. **초기 성능 튜닝:** 파이프라인이 효율적이고 비용 효과적인지 확인하기 위해 개발 초기 단계부터 성능 튜닝을 수행합니다.
7. **인프라 효율성:**
 - **스토리지 수명 주기 (Storage Lifecycle):** 데이터 유형에 맞는 적절한 스토리지를 사용합니다. 자주 사용되지 않는 데이터는 저비용 스토리지로 이동하여 비용을 절감합니다.
 - **클러스터 자동 종료 (Auto-termination of Clusters):** 사용되지 않는 클러스터는 자동으로 종료되도록 설정하여 불필요한 리소스 낭비를 방지합니다.
8. **보안 및 비즈니스 연속성:** 데이터와 모델 자체의 보안을 보장하고, 모델이 비즈니스 요구사항과 일치하며 노후화되지 않도록 관리합니다.
9. **관측 가능성 (Observability):** 파이프라인과 생성된 모델이 제대로 작동하는지 모니터링할 수 있어야 합니다. 또한, 사용자 사용량, 성능, 지연 시간 등을 모니터링하여 문제 발생 시 신속하게 대응할 수 있도록 합니다.
10. **자동화 및 CI/CD 투자:** 인프라를 코드로 관리(Infrastructure as Code)하여 반복 가능성을 높이고, 자동화된 테스트를 통해 모델이 예상대로 작동하는지 쉽게 확인할 수 있도록 합니다.
11. **센터 오브 엑셀런스 (Center of Excellence) 구축:** 조직 내 중앙 그룹이 모범 사례를 홍보하고, 견고한 파이프라인 구축에 대한 전문 지식을 제공하며, 방법론을 통합하고 지원을 제공합니다.

3.2 소프트웨어 개발 수명 주기 (SDLC) 단계

데이터 파이프라인 개발에는 여러 단계가 있으며, 각 단계는 특정 목표와 리소스 활용 전략을 가집니다.

1. 개발 단계 (Development Phase)

- **목표:** 요구사항 이해, 비즈니스 로직 구현, 테스트 케이스 생성 및 테스트 수행.
- **리소스:** 일반적으로 단일 노드 클러스터와 같은 소규모 리소스를 사용합니다. 스팟 인스턴스(Spot Instances)는 일반 인스턴스보다 80-90% 저렴하므로 비용 효율적인 개발에 활용될 수 있습니다.

2. 스테이징 단계 (Staging Phase)

- **목표:** 개발 단계를 벗어나 프로덕션 환경으로의 배포를 준비하는 단계입니다.
- **데이터셋:** 프로덕션 환경과 유사하거나 더 큰 규모의 데이터셋을 사용하여 테스트합니다.
- **테스트:** 테스트 범위를 확대하고, 파이프라인을 자동화합니다.
- **소스 제어:** 파이프라인 코드를 Git 또는 다른 소스 제어 시스템에 체크인하여 반복 가능성을 확보합니다.
- **트리거:** 대화형 (Interactive) 트리거에서 자동화된 (Automated) 트리거로 전환을 시작합니다.
- **자격 증명:** 개별 사용자 자격 증명 대신 파이프라인 운영을 위한 서비스 주체 (Service Principals)를 설정하고 사용합니다.

3. 프로덕션 단계 (Production Phase)

- **목표:** 파이프라인과 생성된 모델이 비즈니스 또는 대중에게 사용 가능하도록 배포되는 최종 단계입니다.
- **자동화:** 모든 것이 완전히 자동화됩니다.
- **정책 및 자격 증명:** 올바른 정책이 적용되고, 서비스 주체만 파이프라인에 필요한 자격 증명을 제공하는 데 사용됩니다.

3.3 서비스 수준 계약 (SLA)

SLA는 서비스 제공자와 사용자 간의 서비스 품질에 대한 공식적인 약속입니다. 다양한 워크로드 유형에 따라 SLA 지표가 달라집니다.

1. **배치 (Batch) 워크로드:** 처리해야 할 데이터 볼륨과 전체 처리 시간으로 정의됩니다. 예를 들어, 특정 시간 내에 대량의 데이터를 처리해야 하는 요구사항입니다.
2. **스트리밍 (Streaming) 워크로드:** 초당 트랜잭션 수 (TPS, Transactions Per Second)로 측정됩니다. 파이프라인이 초당 1,000 또는 10,000 트랜잭션을 처리할 수 있어야 하는 것과 같습니다.
3. **비즈니스 인텔리전스 (BI) 워크로드:** 주로 지연 시간 (Latency)으로 측정됩니다. 사용자가 버튼을 클릭한 시점부터 시각화된 결과가 화면에 나타나는 시간 (예: 0.5초 또는 1초)을 의미합니다.
4. **동시성 (Concurrency):** 시스템을 동시에 사용하는 사용자 수를 나타냅니다. 한 명의 사용자가 시스템을 사용하는 것보다 천 명의 사용자가 동시에 상호작용하는 것이 훨씬 더 많은 인프라를 요구합니다.
5. **가용성 (Availability):** 시스템이 얼마나 자주 다운되는지, 또는 반대로 얼마나 오랫동안 사용 가능한 상태를 유지하는지를 나타냅니다.
 - **계산:** $(\text{총 가동 시간}) / (\text{총 가동 시간} + \text{총 다운 시간}) \times 100\%$
 - **예시:** 99.999% 가용성은 '5-Nines'라고 불리며, 1년에 단 5분만 다운타임을 허용하는 매우 엄격한 기준입니다. 99.99% (4-Nines)는 조금 더 여유로운 기준을 제공합니다. 비즈니스 요구사항에 따라 허용 가능한 다운타임이 결정됩니다.

6. **재해 복구 (Disaster Recovery):** 시스템에 문제가 발생했을 때 얼마나 빨리 복구할 수 있는지를 나타냅니다.

- **RTO (Recovery Time Objective):** 시스템을 기능적인 상태로 복원하는 데 걸리는 목표 시간입니다. 예를 들어, RTO가 15분이라면 시스템은 15분 이내에 다시 작동해야 합니다.
- **RPO (Recovery Point Objective):** 데이터를 신뢰할 수 있는 백업 시점으로 되돌릴 수 있는 최대 시간입니다. 예를 들어, RPO가 15분이라면 최대 15분 전의 데이터까지 복구할 수 있어야 합니다.
- **고가용성 (High Availability):** 데이터 센터 다운과 같은 문제를 피하기 위해 다중 지역 (Multi-region) 시스템을 사용하여 한 지역이 다운되더라도 다른 지역에서 시스템이 계속 작동하도록 할 수 있습니다.

3.4 비용 절감 전략

데이터 플랫폼 운영 비용을 효과적으로 관리하고 절감하는 것은 매우 중요합니다.

1. **플랫폼 선택:** 최고의 가격 대비 성능을 제공하면서도 적절한 제어 기능을 제공하는 플랫폼을 선택해야 합니다. 성능, 비용, 제어 사이의 균형을 찾는 것이 중요합니다.
2. **업계 벤치마크 활용:** 플랫폼 선택에 도움이 되는 두 가지 중요한 업계 벤치마크가 있습니다.
 - **TPC-DS (Transaction Processing Performance Council - Decision Support):** 분석 및 의사결정 지원을 위한 벤치마크입니다. 데이터 웨어하우스에서 데이터를 얼마나 빨리 쿼리할 수 있는지를 측정합니다. BI 지연 시간 및 동시성과 관련이 있습니다. Databricks가 이 분야에서 기록을 세웠습니다.
 - **TPC-DI (Transaction Processing Performance Council - Data Integration):** 데이터 통합 벤치마크입니다. 플랫폼이 데이터 웨어하우스를 얼마나 빨리 구축하고 데이터를 로드할 수 있는지를 나타냅니다. 데이터 수집 프로세스 시간을 최소화하는 것이 목표입니다.
3. **가정하지 말고 검증:** 성능 요구사항에 대한 가정을 세우기보다는 실험을 통해 검증하는 시간을 가져야 합니다.
4. **코드 최적화:** 더 효율적인 코드를 작성하여 시스템 효율성을 높입니다.
 - **데이터프레임 API 활용:** 자체 개발 UDF(User Defined Functions) 대신 고도로 최적화된 데이터 프레임 API를 활용합니다. UDF는 Python 엔진 시작 등 비효율적인 요소를 포함할 수 있습니다.
 - **ML 훈련 중 캐싱:** ML 훈련 시 캐싱을 사용하여 이전에 계산된 값과 데이터 구조를 재사용하여 성능을 크게 향상시킵니다.
 - **바이너리 형식 사용:** Delta 또는 Parquet과 같은 바이너리 형식을 사용하여 데이터 크기를 줄이고 효율성을 높입니다. CSV와 같은 텍스트 파일은 매번 전체 레코드를 읽어야 하지만, 바이너리 형식은 더 효율적이고 빠르며 비용을 절감합니다.
5. **인프라 최적화:**
 - **적절한 인프라 사용:** 인스턴스 유형, 크기, 클러스터 내 인스턴스 수 등 적절한 인프라를 사용합니다. 너무 작으면 성능이 저하되고, 너무 크면 비용이 많이 듭니다.
 - **거버넌스:** 클러스터 크기(인스턴스 수) 및 인스턴스 유형에 대한 거버넌스 정책을 수립하여 불필요하게 큰 클러스터 생성을 방지합니다. 환경 태그를 사용하여 클러스터 유형을 지정할 수 있습니다.
 - **오토스케일링 (Autoscaling):** 현재 요구사항에 따라 클러스터가 자동으로 확장 및 축소되도록 설정합니다. 컴퓨팅 수요가 증가하면 확장하고, 수요가 줄어들면 축소하여 비용을 절감합니다.
 - **자동 종료 (Auto-termination):** 사용자가 클러스터를 끄는 것을 잊었을 경우, 일정 시간(예: 30

분) 비활성 상태 후 클러스터가 자동으로 종료되어 리소스를 해제합니다. 주말 동안 실행되는 클러스터로 인한 수천 달러의 비용 낭비를 방지할 수 있습니다.

- **스팟 인스턴스 (Spot Instances):** 인스턴스 비용을 최대 90%까지 절감할 수 있습니다. 미션 크리티컬한 작업에는 권장되지 않지만, 모델 훈련과 같은 작업에는 매우 효과적입니다. (클라우드 제공자가 컴퓨팅 자원이 필요할 경우 인스턴스를 회수할 수 있습니다.)

6. **최신 Spark 버전 사용:** Spark의 각 업데이트에는 버그 수정 및 많은 성능 향상이 포함됩니다. 최신 버전의 Spark를 사용하면 더 나은 품질, 더 많은 기능, 더 높은 효율성을 얻을 수 있습니다.
7. **서버리스 (Serverless)로 전환:** 클러스터 사용에서 서버리스 환경으로 전환하는 것을 고려합니다. 서버리스는 사용한 만큼만 비용을 지불하며, 클러스터는 컴퓨팅을 수행하지 않아도 실행 중인 동안 비용이 발생합니다. 서버리스는 클러스터 시작 시간(1분)보다 훨씬 빠른 5초 이내에 시작되는 장점도 있습니다.
8. **모니터링 및 경고 (Monitoring and Alerting):** 오랫동안 실행되거나 과도하게 큰 클러스터를 감지하고, 많은 비용이 발생하기 전에 클러스터를 중지할 수 있도록 모니터링 및 경고 시스템을 구축합니다. 태그를 사용하여 리소스 사용자를 식별하면, 데이터 센터 청구서에서 어떤 프로젝트, 팀 또는 애플리케이션이 비정상적인 지출을 유발하는지 쉽게 파악할 수 있습니다.

4 지속적인 통합 및 배포 (CI/CD)

개념

CI/CD 개요 CI/CD는 소프트웨어 개발의 핵심적인 방법론으로, 코드 변경 사항을 자동으로 통합(Continuous Integration)하고, 테스트를 거쳐 배포(Continuous Delivery/Deployment)하는 과정을 의미합니다. 이를 통해 개발 속도를 높이고, 오류를 조기에 발견하며, 안정적인 소프트웨어 릴리스를 가능하게 합니다.

4.1 CI/CD의 기본 개념

1. **지속적인 통합 (Continuous Integration, CI):** 개발자가 코드를 변경하고 Git 저장소에 체크인(commit)하는 즉시, 자동화된 빌드 및 테스트가 트리거됩니다. 이 과정이 성공하면, 코드는 자동으로 다음 단계(예: 스테이징 환경)로 푸시될 수 있습니다. 만약 실패하면 개발자에게 즉시 알림이 전송되어 문제를 수정할 수 있도록 합니다.
2. **지속적인 배포 (Continuous Delivery/Deployment, CD):** CI 단계를 통과하고 스테이징 환경에서 테스트를 완료한 코드는 배포 준비가 된 상태가 됩니다.
 - **Continuous Delivery:** 코드가 프로덕션 환경에 배포될 준비가 되었지만, 실제 배포는 수동 승인을 통해 이루어집니다.
 - **Continuous Deployment:** 모든 회귀 테스트(Regression Testing)를 통과하면 코드가 완전히 자동으로 프로덕션에 배포됩니다.

Figure 1: CI/CD 워크플로우 (개념도)

CI/CD 워크플로우: 개발자가 변경 사항을 커밋하면, 빌드가 자동으로 수행되고 테스트가 실행됩니다. 성공하면 스테이징 환경으로 이동하여 추가 테스트를 거치고, 이 또한 성공하면 프로덕션 환경으로 마이그레이션됩니다. 이 과정은 '코드(Code) → 빌드(Build) → 릴리스(Release) → 배포(Deploy) → 테스트(Test) → 운영(Operate)'의 흐름을 따릅니다.

주의사항

자동 롤백/페일오버 스테이징 또는 프로덕션 환경에 새로운 버전을 배포할 때 문제가 발생할 경우를 대비하여, 자동으로 이전 버전으로 롤백하거나 페일오버할 수 있는 메커니즘을 갖추는 것이 중요합니다.

4.2 데이터 플랫폼에서의 CI/CD

Databricks와 같은 데이터 플랫폼에서 CI/CD를 구현할 때는 다음과 같은 사항을 고려합니다.

1. **다중 환경 관리:** 개발(Development), 스테이징(Staging), 프로덕션(Production)과 같은 여러 환경을 관리합니다. Databricks에서는 이러한 환경이 '워크스페이스(Workspaces)'에 느슨하게 매핑됩니다.
2. **로컬 개발 및 버전 관리:** 개발자는 IDE를 사용하여 로컬에서 개발하고, 개발된 아티팩트(artifacts)를 버전 관리 시스템(예: Git)에 체크인합니다. 일반적인 SDLC 관행(체크아웃, 병합, 푸시, 커밋)을 따릅니다.
3. **아티팩트 이동:** CI/CD 프로세스는 개발된 아티팩트를 한 환경에서 다른 환경으로 이동시키는 역할을 합니다(예: 개발 → 스테이징 → 프로덕션).

4. **공유 라이브러리 (Wheel Files):** 고수준 로직에는 노트북이 사용되지만, 팀 간에 공유되는 라이브러리 등은 일반적으로 '휠 파일(wheel files)' 형태로 개발됩니다. 이 휠 파일은 CI 단계에서 빌드 서버에서 한 번 빌드된 후, 아티팩토리(Artifactory)나 저장소에 저장되어 배포 단계에서 여러 환경에 푸시될 수 있습니다. 이는 환경마다 빌드를 반복하는 오버헤드를 방지합니다.

5 인프라 자동화: Infrastructure as Code (IaC)

개념

Infrastructure as Code (IaC) 개요 IaC는 클라우드 인프라(VM, 네트워크, 보안 그룹 등)와 데이터 플랫폼(워크스페이스)을 수동 클릭이나 스크립트 대신 코드로 정의하고 관리하는 접근 방식입니다. 이를 통해 인프라 구축 및 변경을 반복 가능하고 일관되며 예측 가능하게 만듭니다.

5.1 IaC의 핵심 원칙

1. **모든 것을 코드로 정의:** 구축하려는 모든 인프라 요소를 코드로 작성합니다. 이는 AWS 콘솔이나 Azure 포털에서 수동으로 클릭하거나 자체 스크립트를 사용하는 대신, 표준화된 코드를 통해 인프라를 생성하고 관리하는 것을 의미합니다.
2. **지속적인 테스트 및 배포:** 진행 중인 모든 작업을 지속적으로 테스트하고 배포합니다. 이는 CI/CD 파이프라인과 통합되어 인프라 변경 사항도 코드 변경 사항처럼 관리됩니다.
3. **작고 독립적인 변경 단위:** 독립적으로 변경할 수 있는 작고 단순한 구성 요소를 구축합니다. 인프라의 특정 부분을 변경해야 할 경우, 해당 구성 요소를 제어하는 코드만 수정하고 재배포하여 인플레이스(in-place) 변경을 수행할 수 있습니다.
4. **클라우드 인프라 및 워크스페이스 인프라:** Databricks와 같은 데이터 플랫폼의 경우, IaC는 클라우드 인프라(예: VM, VNET, 보안 그룹)뿐만 아니라 Databricks 워크스페이스 자체의 인프라(예: 클러스터, 작업, 노트북 설정)에도 적용될 수 있습니다.

5.2 자동화 도구 비교: 구성 관리 vs. 프로비저닝

IaC 도구는 크게 '구성 관리(Configuration Management)'와 '프로비저닝(Provisioning)' 두 가지 유형으로 나눌 수 있습니다.

Table 2: 구성 관리 도구와 프로비저닝 도구 비교

특징	구성 관리 (Configuration Management)	프로비저닝 (Provisioning)	예시
목적	이미 프로비저닝된 인프라 내 소프트웨어 컴포넌트 변경 (예: 라이브러리 업데이트)	인프라를 처음부터 구축하거나 변경 (예: VM, 네트워크 생성)	Chef, Puppet, Ansible (구성 관리) Terraform, CloudFormation, Bicep (프로비저닝)
변경 방식	가변적 (Mutable): 인플레이스(in-place) 변경	불변적 (Immutable): 스크래치부터 재배포	
특성	절차적(Procedural): 특정 언어(JavaScript, Ruby) 사용	선언적(Declarative): 원하는 최종 상태를 정의	Terraform은 HCL(HashiCorp Configuration Language) 사용
에이전트	에이전트 기반: 변경 적용을 위해 머신에 에이전트 필요	에이전트리스: API 레이어가 서비스 레이어 역할	

- **구성 관리 (Configuration Management):** Chef, Puppet, Ansible과 같은 도구들이 여기에 속합니다. 이미 구축된 인프라 내에서 소프트웨어 컴포넌트(예: 라이브러리)를 변경하는 데 사용됩니다. 이는 '가변적(mutable)'이며, 인플레이스(in-place) 변경을 수행합니다. 일반적으로 절차적(procedural)이며, 변경을 적용하기 위해 대상 머신에 에이전트가 필요합니다.
- **프로비저닝 (Provisioning):** Terraform, CloudFormation(AWS), Bicep(Azure)과 같은 도구들이 여기에 속합니다. 인프라를 처음부터 구축하거나 대규모 변경을 수행하는 데 사용됩니다. 이는 '불변적(immutable)'이며, 변경 시에는 스크래치부터 다시 배포하는 경향이 있습니다. 일반적으로 선언적(declarative)이며, 에이전트가 필요 없이 API 레이어를 통해 작동합니다.
- **Databricks와 Terraform:** Databricks는 Terraform과 강력한 파트너십을 맺고 있으며, Databricks의 거의 모든 측면(워크스페이스, 인프라)을 Terraform을 사용하여 자동화할 수 있습니다. Terraform은 클라우드 불가지론적(cloud agnostic)이며, 코드 재사용성, 반복 가능성, 예측 가능성이 뛰어납니다. 또한 REST API를 지원하여 사용자 정의 래퍼를 구축할 수 있습니다. Terraform은 Databricks

워크스페이스뿐만 아니라 그 기반이 되는 클라우드 인프라(가상 네트워크, 서브넷, 보안 그룹 등)도 자동화할 수 있습니다.

5.3 Terraform 데모 예시

Terraform은 선언적 스크립트를 사용하여 인프라를 정의합니다.

□ 예제

Terraform 스크립트 예시 (Azure 리소스 생성) 아래는 Azure에서 가상 네트워크(VNET)를 생성하는 Terraform 스크립트의 일부입니다. 이 스크립트는 구독, 지역, DNS 규칙, 공용 IP 사용 여부 등을 선언적으로 정의합니다.

```

1 resource "azurerm_virtual_network" "example" {
2   name                = "example-vnet"
3   address_space       = ["10.0.0.0/16"]
4   location             = azurerm_resource_group.example.location
5   resource_group_name = azurerm_resource_group.example.name
6   dns_servers          = ["10.0.0.4", "10.0.0.5"]
7   # public_ip_enabled = false 예시에서( 언급된 public IP 제어)
8 }

```

Listing 1: Terraform Azure VNET 생성 예시

1. **스크립트 구성:** `provider.tf`와 같은 파일에 인프라 정의 스크립트를 작성합니다. Terraform은 `TF state`라는 상태 파일을 자동으로 생성하여, 이전에 생성된 인프라의 상태를 추적하고 관리합니다. 이를 통해 부분적인 변경이나 업데이트가 가능하며, 전체를 처음부터 다시 배포할 필요 없이 필요한 부분만 수정할 수 있습니다.

2. Terraform 명령:

- **terraform plan:** 스크립트를 실행하기 전에 변경될 내용을 미리 보여주는 드라이 런(dry run)을 수행합니다. 추가될 항목, 변경될 항목, 파괴될 항목 등을 보고서 형태로 제공합니다.
- **terraform apply:** 실제 스크립트를 실행하여 인프라를 구축하거나 변경합니다. 이 명령을 실행하면 Azure 리소스와 Databricks 리소스가 생성되어 워크스페이스가 준비됩니다.

이러한 과정을 통해 인프라 구축이 고도로 반복 가능하고 표준화됩니다.

6 워크스페이스 조직 및 데이터 메시

개념

워크스페이스 조직 및 데이터 메시 개요 데이터 플랫폼에서 '워크스페이스'는 격리 경계를 제공하는 중요한 단위입니다. 조직의 규모와 복잡성이 커짐에 따라 워크스페이스를 효과적으로 조직하는 전략이 필요하며, '데이터 메시'는 데이터를 제품처럼 취급하여 분산된 데이터 관리를 가능하게 하는 아키텍처 개념입니다.

6.1 워크스페이스 조직 (Workspace Organization)

Databricks와 같은 데이터 플랫폼에서 '워크스페이스'는 격리 경계를 제공하는 핵심 단위입니다.

- 격리 경계:** 워크스페이스는 자체 클러스터, 작업, 노트북, 폴더 구조를 가지며, 다른 워크스페이스와 격리됩니다. 이는 특정 URL로 접근할 수 있는 독립적인 환경으로 볼 수 있습니다. 예를 들어, 개발, 스테이징, 프로덕션을 위한 세 개의 다른 워크스페이스(URL)를 가질 수 있습니다.
- 환경 진화:** 전통적으로는 개발(Dev), 스테이징(Staging), 프로덕션(Production)의 세 가지 환경만 가졌지만, 이제는 비즈니스 라인(Line of Business, LOB)이나 프로젝트별로 세분화된 격리 환경을 요구하는 경우가 많습니다.
 - 예를 들어, LOB 1은 자체 개발, 스테이징, 프로덕션 환경을 가질 수 있으며, LOB 1 내의 여러 프로젝트도 자체 격리 전략을 원할 수 있습니다.
 - 성숙한 조직에서는 수백에서 수천 개의 워크스페이스를 운영하는 것이 드물지 않습니다.
- 중앙 집중식 사용자 관리:** 이러한 격리를 구동하는 데 필요한 사용자, 그룹, 서비스 주체 등은 Active Directory 또는 Intra와 같은 중앙 집중식 SSO(Single Sign-On) 시스템에서 관리됩니다. 기업 수준에서 사용자 및 그룹을 프로비저닝하고, 권한이 있는 사용자만 특정 프로젝트 워크스페이스에 로그인할 수 있습니다.
- 공유 프로젝트:** 두 개 이상의 LOB가 공통 데이터 도메인을 공유하는 경우, 해당 LOB를 아우르는 공유 프로젝트 워크스페이스를 가질 수도 있습니다.
- 표준화된 프로비저닝의 중요성:** 수천 개의 워크스페이스를 수동으로 배포하는 것은 불가능하므로, 반복 가능하고 표준화된 프로비저닝 방식(IaC)이 매우 중요합니다. 이는 워크스페이스뿐만 아니라 데이터 플랫폼의 데이터가 저장되는 스토리지 계정(Storage Accounts)에도 적용되어야 합니다. 임의의 스토리지 계정 생성을 방지하고, 표준화된 계층 구조를 통해 스토리지를 관리해야 합니다.

6.2 데이터 메시 (Data Mesh)

데이터 메시는 데이터를 조직의 인프라나 프로젝트가 아닌 '제품'으로 취급하는 개념입니다.

- 데이터를 제품으로 취급:** 데이터 메시의 핵심은 데이터를 그 자체로 가치를 지닌 제품으로 간주하는 것입니다.
- Databricks와 데이터 메시:** Databricks와 같은 데이터 플랫폼은 데이터 메시 아키텍처에 매우 잘 부합합니다. 워크스페이스는 데이터 도메인을 생성할 수 있는 가장 낮은 수준의 격리 경계를 형성합니다.
- 데이터 메시의 4가지 특징:**
 - 잘 조직된 데이터 도메인 (Well-organized Data Domains):** 마케팅, 영업, 공급망과 같이 잘

정의된 데이터 도메인을 가집니다.

- **데이터 소유자 및 연합 거버넌스 (Data Owners and Federated Governance):** 각 데이터 도메인은 자체 데이터 소유자를 가지며, 데이터 거버넌스에 대한 책임을 집니다.
- **인프라 관점의 셀프 서비스 (Self-served from an Infrastructure Perspective):** 데이터 소유자가 자체 데이터 도메인을 생성할 수 있도록 인프라 프로비저닝이 표준화되어 셀프 서비스가 가능해야 합니다.
- **Unity Catalog:** Databricks의 Unity Catalog는 이러한 데이터 도메인들을 묶어주는 '접착제' 역할을 합니다. 서로 다른 도메인에서 생성된 데이터를 결합하여 최종 데이터 제품을 형성할 수 있도록 돕습니다.

4. **메달리온 아키텍처와의 연관성:** 데이터 플랫폼 관점에서 메달리온 아키텍처 (Bronze → Silver → Gold)의 골드(Gold) 레이어는 여러 데이터 도메인에서 파생된 데이터 제품으로 볼 수 있습니다.

7 Databricks에서의 CI/CD 및 MLOps

개념

Databricks CI/CD 및 MLOps Databricks는 소스 코드 제어 시스템과의 긴밀한 통합을 통해 CI/CD 및 MLOps를 지원합니다. 이를 통해 데이터 파이프라인과 ML 모델의 개발, 배포, 운영을 효율적으로 자동화하고 관리할 수 있습니다.

7.1 CI/CD 파이프라인 (Azure DevOps 예시)

CI/CD 또는 DevOps는 소스 코드 제어 시스템(예: GitHub, Bitbucket)과 DevOps 시스템(예: Azure DevOps, Jenkins)의 조합으로 구현됩니다.

1. **파이프라인 구성:** 전통적으로는 빌드 파이프라인(Build Pipeline)과 릴리스 파이프라인(Release Pipeline)으로 나누었지만, 최근에는 단일 스테이지 접근 방식(Single Stage Approach)으로 통합되는 추세입니다.
 - **빌드 파이프라인:** 단위 테스트, 통합 테스트를 실행하고, 휠 파일(wheel files)과 같은 라이브러리를 빌드합니다.
 - **릴리스 파이프라인:** 특정 액션(예: 소스 코드 제어 시스템의 변경) 또는 수동 게이팅(Manual Gating)에 따라 빌드된 자산을 한 환경에서 다른 환경으로 이동시킵니다.
2. **자동화된 트리거:** 소스 코드 제어 시스템에 코드를 푸시하면 빌드 파이프라인이 자동으로 트리거되어 일련의 단계를 실행합니다. 이는 코드가 변경되는 즉시 테스트를 수행하여 문제를 조기에 발견하고, 소프트웨어 릴리스 주기를 단축시킵니다.
3. **API 지원:** Databricks와 같은 데이터 플랫폼은 아티팩트를 내보내고 가져오는 데 필요한 강력한 API를 제공합니다. 이를 통해 DevOps 빌드 서버는 Databricks 플랫폼과 상호작용하여 자산을 빌드하고, 테스트하며, 다른 환경으로 푸시할 수 있습니다.
4. **클러스터 관리:** 개발 환경에서는 대화형(Interactive) 클러스터를 사용하는 반면, 프로덕션 환경에서는 자동화된 작업 클러스터(Automated Job Cluster)를 사용하는 것이 일반적입니다. CI/CD 프로세스 중에 이러한 클러스터 유형 간의 전환이 이루어집니다.

7.2 MLOps (Machine Learning Operations)

MLOps는 ML 모델의 개발, 배포, 운영을 위한 DevOps 원칙을 적용한 것입니다.

1. **MLOps 특징:** 실험 관리, 피쳐 스토어(Feature Stores), 모델 레지스트리(Model Registries)와 같은 기능을 포함합니다. 모델 레지스트리에서는 모델을 '챌린저(Challenger)'에서 '챔피언(Champion)'으로 승격시키는 등의 관리가 이루어집니다.
2. **모델 배포 전략:**
 - **모델 이동:** 모델을 한 번 훈련한 후, 보안 또는 레지스트리 기능을 사용하여 모델을 한 환경에서 다른 환경으로 이동시키는 방법입니다. 하지만 훈련 데이터가 특정 환경에 종속될 수 있으므로 항상 이상적이지는 않습니다.
 - **훈련 코드 이동:** 훈련 코드를 환경 간에 이동시킨 후, 각 환경에서 모델을 다시 훈련하는 방법입니다. 이는 환경별 데이터 특성을 고려할 때 더 적합할 수 있습니다.
3. **Databricks의 MLOps 지원:** Databricks는 폴더 구조, API를 통한 소스 코드 제어 시스템과의 연동

등을 통해 MLOps를 강력하게 지원합니다. MLOps는 개발 초기부터 고려되어야 하며, 플랫폼 자체에서 강력한 지원을 제공해야 합니다.

8 관측 가능성 (Observability) 및 모니터링

개념

관측 가능성 및 모니터링 개요 데이터 플랫폼의 관측 가능성과 모니터링은 시스템의 상태를 이해하고, 문제 발생 시 신속하게 진단하며, 데이터 품질을 보장하는 데 필수적입니다. Databricks는 시스템 테이블, 대시보드, AI 기반 이상 감지, 레이크하우스 모니터링, DQX와 같은 다양한 도구를 제공합니다.

8.1 시스템 테이블 (System Tables)

Databricks는 플랫폼 내에서 발생하는 다양한 메타데이터와 사용량 정보를 '시스템 테이블' 형태로 제공합니다. 이 테이블들은 Unity Catalog의 `system` 카탈로그 아래에 위치합니다.

1. 활용 분야:

- **빌링 (Billing):** `system.billing` 스키마에서 사용량 정보를 확인하여, 조직 내 다양한 부서나 프로젝트에 비용을 청구(chargeback)하는 데 활용할 수 있습니다. SKU, 사용 시작/종료 시간, 사용자 정의 태그(Custom Tags), 사용량 등을 상세하게 제공합니다. 사용자 정의 태그는 사용량을 세분화하여 특정 프로젝트나 팀의 지출을 파악하는 데 매우 중요합니다.
- **컴퓨트 (Compute):** `system.compute` 스키마에서 클러스터 사용 정보를 확인할 수 있습니다. 클러스터 이름, 소유자, 생성/삭제 시간, 노드 유형(예: AWS R5X large), 메모리/CPU 사용량 등을 통해 클러스터의 효율적인 사용 여부를 모니터링할 수 있습니다.
- **감사 (Auditing):** `system.access` 스키마에서 누가 어떤 데이터에 언제 접근했는지, 무엇이 삭제되었는지 등의 감사 정보를 확인할 수 있습니다. 이는 보안 및 규정 준수에 매우 중요합니다.

2. **강점:** 시스템 테이블은 매우 세분화된 원격 측정(telemetry) 데이터를 제공하여, 관리자가 워크스페이스를 모니터링하고 다양한 분석을 수행할 수 있도록 합니다.

8.2 대시보드 (Dashboards)

Databricks는 시스템 테이블을 기반으로 하는 다양한 내장 대시보드를 제공하며, 사용자는 이를 통해 시각적으로 시스템 상태